# Introduction to Program Testing with the BATS Framework

Christoph Niethammer

Testing software is not optional –
it is mandatory;
Failing to test is an act of neglicence

# Why testing is mandatory

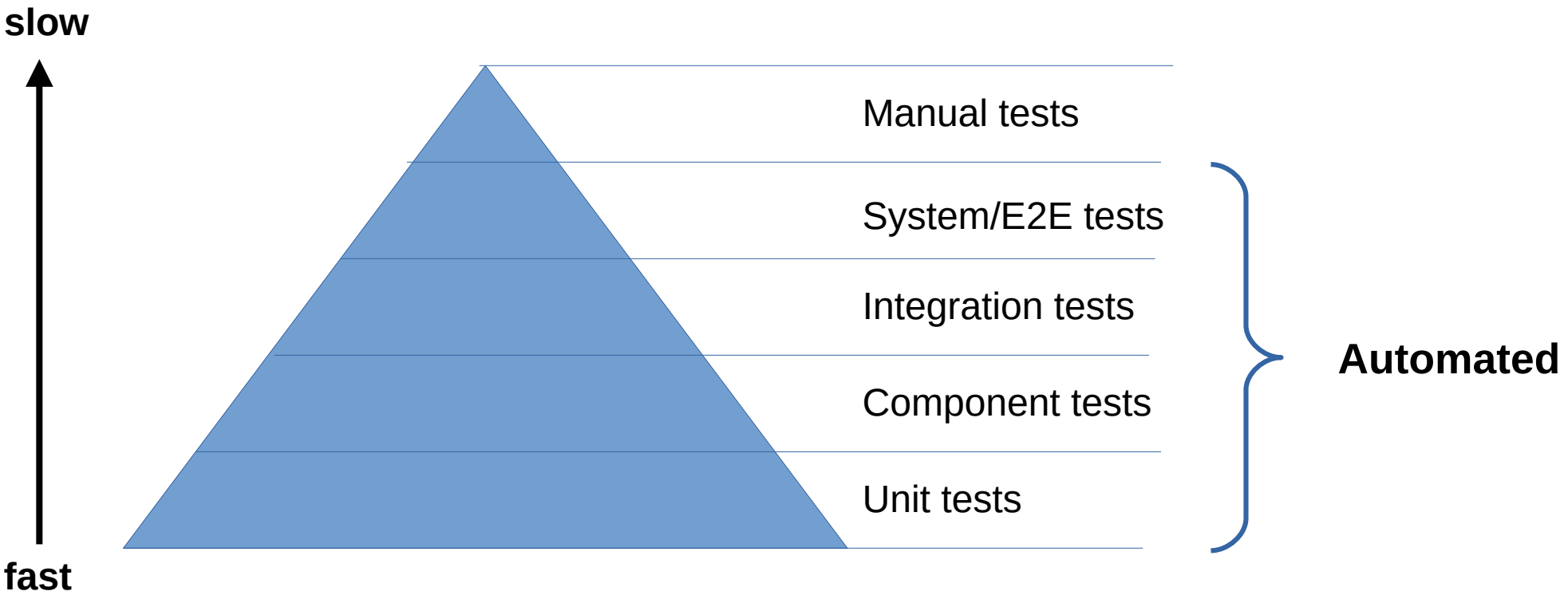
- Skipping tests means delaying the discovery of defects
- Finding defects later makes them more expensive to fix
- Unverified software introduces risks:
  - System outages
  - Security vulnerabilities
  - Poor user experience
  - Loss of business credibility

„Optimism is an occupational hazzard of programming: feedback is the treatmen.“
(Kent Beck)

„A program that has not been tested does not work“
(Bjarne Stroustrup)

„If it’s not tested, it’s broken“
(Bruce Eckel)

# The testing pyramide



# Introduction to BATS

28.11.2025, Christoph Niethammer

# What is BATS?

- BATS – Bash Automated Testing System
- Verify that UNIX programs behave as expected
- Simple bash-based test syntax
- Only bash 3.2+ as dependency
- TAP compliant output

# Installation methods

- Via package manager
- Via npm
- From source
- As git submodule in your project
- Running from a docker container

# BATS concept

- Tests are written in *.bats files, which can be organized in directories
- A bats file contains one or more tests marked with @test
- tests get preprocessed into bash functions
- Bats executes
  setup → test → teardown
  for each test

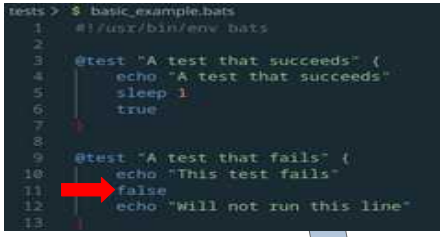
# Basic test example

tests > $ basic_example.bats

```
1   #!/usr/bin/env bats
2
3   @test "A test that succeeds" {
4       echo "A test that succeeds"
5       sleep 1
6       true
7   }
8
9   @test "A test that fails" {
10      echo "This test fails"
11      false
12      echo "Will not run this line"
13  }
```

# Running tests

- Provide bats file or directory with bats files
- Use `-r` option to recurse into subdirectories
- Output shown only for failed tests




```
$ bats  basic_example.bats
basic_example.bats
✓ A test that succeeds
✗ A test that fails
  (in test file basic_example.bats, line 11)
    `false' failed
  This test fails

2 tests, 1 failure
```

# setup and teardown

H L R S

**setup() / teardown()**

Can be used to initalize and clean up before and after each test is executed

**Example:** Use relative path to known bats test file location (`$BATS_TEST_FILENAME`) to initalize `PATH` variable for execution of the hello.sh script in the test

tests > $ setup_example.bats

```
1  #!/usr/bin/env bats
2
3  setup() {
4      DIR="$( cd "$( dirname "$BATS_TEST_FILENAME" )" >/dev/null 2>&1 && pwd )
5      # make executables in bin/ visible to PATH
6      PATH="$DIR/../bin:$PATH"
7  }
8
9  @test "Test 1" {
10     hello.sh
11 }
```

# Skipping tests

- Skip test with skip command in a test
- (optionally) provide a reason

```
tests > $ skip_example.bats
1  #!/usr/bin/env bats
2
3  @test "A skipped test" {
4      skip "Skipping test that would fail"
5      false
6  }
```

```
$ bats tests/skip_example.bats
skip_example.bats
- A skipped test (skipped: Skipping test that would fail)


1 test, 0 failures, 1 skipped
```

# The run command

- Allows to test, failing commands
- invokes a command in a subshell
- saves the exit status and output into special global variables `$status`, `$output`, and `$BATS_RUN_COMMAND`
- returns with a 0 status

```
tests > $ run_example.bats
1  #!/usr/bin/env bats
2
3  @test "invoking cat with a nonexistent file prints an error" {
4    run cat nonexistent_file
5    [ "$status" -eq 1 ]
6    [ "$output" = "cat: nonexistent_file: No such file or directory" ]
7    [ "$BATS_RUN_COMMAND" = "cat nonexistent_file" ]
8  }
```

```
$ bats tests/run_example.bats

run_example.bats

✓ invoking cat with a nonexistent file prints an error

1 test, 0 failures
```

# Run command and pipes

- Use **`bats_pipe`** helper when pipes are used in a run command
- requires the usage of `\|` not `|`
- By default mimics `set -o pipefail`, returning the status of the last (rightmost) command that exits with a non-zero status

```
tests > $ bats_pipe_example.bats
  1  #!/usr/bin/env bats
  2
  3  @test "Invoking 'ls piped to cat'" {
  4  |  run bats_pipe ls \| cat
  5  |  [ "$status" -eq 0 ]
  6  }
```

# Bats support libraries

- **bats-support**
  common functions to test helper libraries
  (e.g., error reporting, output formating, execution restrictions)
- **bats-assert**
  common assertions
  (e.g., exit status, output content, regex matching)
- **bats-file**
  common filesystem related assertions and helpers
  (e.g., file existence, file attribtes, file content)

# Assertions

Assertions perform a test and return `1` on failure or `0` on success

- assert_success / assert_failure
  → check $status
- assert_output / refute_output
  → check $output
  possible options:
  - --partial
  - --regex

tests > $ assert_example.bats

```
1  #!/usr/bin/env bats
2
3  setup() {
4      bats_load_library "bats-support"
5      bats_load_library "bats-assert"
6  }
7
8  @test "A test that uses assertions" {
9      run echo "A test using assertions"
10     assert_success
11     assert_output "A test using assertions"
12 }
13
14 @test "A test that asserts command failure" {
15     run ls nonexistent_file
16     assert_failure
17 }
```

# bats-file assertions

Checks for existence and properties of files and directories

- assert_file_exists
- assert_dir_exists

- assert_file_contains <regex> <engine>
  engines: grep, egrep, pcregep

tests > $ assert_file.bats

```
1  #!/usr/bin/env bats
2
3  setup() {
4      bats_load_library "bats-support"
5      bats_load_library "bats-file"
6  }
7
8  @test "Check if passwd is there" {
9      assert_file_exists /etc/passwd
10 }
11
12 @test "Check if my user is in passwd" {
13     assert_file_contains /etc/passwd $USER
14 }
```

# Test organisation

- tests in *.bats files
  → tests that shoud normally run together
- Grouping *.bats files into directories
  → grouping tests that may run together
- Tagging test
  → fine grained controll for test filtering

# Taging tests

- Each test has list of tags attached to it
- Defined in two ways:
  - Per test using
    `# bats test_tags=<list>`
  - All tests in a *.bats file after the directive
    `# bats file_tags=<list>`

tests > $ tag_example.bats

```
1  #!/usr/bin/env bats
2
3  @test "Test with no tags" {
4      echo "Test 1"
5  }
6
7  # bats file_tags=A
8
9  # bats test_tags=B
10 @test "Test with tags A and B" {
11     echo "Test 2"
12 }
13
14 # bats test_tags=C
15 @test "Test with tags A and C" {
16     echo "Test 3"
17 }
18
19 # bats test_tags=B,C
20 @test "Test with tags A, B, and C" {
21     echo "Test 3"
22 }
```

# Filtering execution

Use `--filter-tags option <list>`

tests > $ tag_example.bats

```
1  #!/usr/bin/env bats
2
3  @test "Test with no tags" {
4      echo "Test 1"
5  }
6
7  # bats file_tags=A
8
9  # bats test_tags=B
10 @test "Test with tags A and B" {
11     echo "Test 2"
12 }
13
14 # bats test_tags=C
15 @test "Test with tags A and C" {
16     echo "Test 3"
17 }
18
19 # bats test_tags=B,C
20 @test "Test with tags A, B, and C" {
21     echo "Test 3"
22 }
```

--filter-tags A

✓ Test with tags A and B
✓ Test with tags A and C
✓ Test with tags A, B, and C

--filter-tags A,B,C

✓ Test with tags A, B, and C

--filter-tags A,!B --filter-tags B,C

✓ Test with tags A and C
✓ Test with tags A, B, and C

# Focusing on a new test

- Special tag **`bats:focus`**
- All other tests are filtered out
- Exit code of succesfull runs overwritten to 1

Note:
Good for testing new tests

```
5   # bats test_tags=B
6   @test "Test with tags A and B" {
7       echo "Test 1"
8   }
9
10  # bats test_tags=bats:focus
11  @test "Test with focus tag" {
12      echo "Test 2"
13  }
```

```
WARNING: This test run only contains teststagged `bats:focus`!
focus_tag_example.bats
Marking test run as failed due to `bats:focus` tag. (Set
`BATS_NO_FAIL_FOCUS_RUN=1` to disable.)
✓ Test with focus tag

1 test, 0 failures
```

# Special Variables

| | |
|---|---|
| BATS_TMPDIR | base temporary directory used by bats to create its temporary files / directories. (default: $TMPDIR. If $TMPDIR is not set, /tmp is used.) |
| BATS_SUITE_TMPDIR | a temporary directory common to all tests of a suite. Could be used to create files required by multiple tests. |
| BATS_FILE_TMPDIR | a temporary directory common to all tests of a test file. Could be used to create files required by multiple tests in the same test file. |
| BATS_TEST_TMPDIR | a temporary directory unique for each test. Could be used to create files required only for specific tests. |
| BATS_TEST_FILENAME | the fully expanded path to the Bats test file. |
| BATS_TEST_DIRNAME | the directory in which the Bats test file is located. |
| BATS_TEST_RETRIES | maximum number of additional attempts that will be made on a failed test before it is finally considered failed. The default of 0 means the test must pass on the first attempt. |
| BATS_TEST_TIMEOUT | the number of seconds after which a test (including setup) will be aborted and marked as failed. Updates to this value in setup() or @test cannot change the running timeout countdown, so the latest useful update location is setup_file(). |

# bats::on_failure hook

- Executed only on failures
- Can be defined / used at any place: test, setup, teardown, …

```
$ on_failure_example.bats
1   @test "my awesome test" {
2     simple_test_setup
3     bats::on_failure() {
4       handle_simple_error_case
5     }
6     complex_test_setup
7     bats::on_failure() {
8       print_debug_information_only_on_error
9     }
10    do_actual_tests
11    check_results
12  }
```

# References

- Documentation: https://bats-core.readthedocs.io/
- Code: https://github.com/bats-core/bats-core

# Introduction to Program Testing with the BATS Framework

Christoph Niethammer

**Thank you for your attention!**

